

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
```

```
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

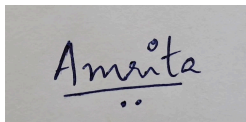
```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I AMRITA ANAIKUMAR - 192471035 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'Amrita' with a small mark below it.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

CO1-AT1

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
<input type="text" name="name">
<input type="email" name="email">
<button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate HTML5 semantic elements missing in the structure.
2. Modify the structure to make it standards-compliant.
3. Interpret the HTTP request generated during form submission.
4. Analyze the HTTP response cycle from server to browser.
5. Suggest improvements for usability and accessibility.

ANSWER:

1. Identify appropriate HTML5 semantic elements missing in the structure.

The missing HTML5 semantic elements are:

- <header>
- <main>
- <section>
- <footer>
- <label> (for form fields)

2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Online Symposium Registration</title>
</head>
```

```
<body>
<header>
<h1>College Symposium Registration</h1>
</header>
<main>
<section>
<form action="/register" method="POST">
<label for="name">Name:</label>
<input type="text" id="name" name="name" required>
<label for="email">Email:</label>
<input type="email" id="email" name="email" required>
<button type="submit">Submit</button>
</form>
</section>
</main>
<footer>
<p>&copy; 2026 College Symposium</p>
</footer>
</body>
</html>
```

3. Interpret the HTTP request generated during form submission.

When the user clicks **Submit**, the browser sends an HTTP **POST** request to the server.

Example:

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=Amrita&email=amrita@example.com

- **POST** → Sends data securely in the request body.
- **/register** → Server URL handling the registration.
- **Host** → Website address.
- **Content-Type** → Specifies the format of submitted data.

4. Analyze the HTTP response cycle from server to browser.

1. User submits the form.
2. Browser sends an HTTP POST request.

3. Server receives and validates the data.
4. Server stores the registration details.
5. Server sends an HTTP response.
6. Browser displays the success page or an error message.

Example response:

HTTP/1.1 200 OK

Content-Type: text/html

Registration Successful!

5. Suggest improvements for usability and accessibility.

- Use <label> for every input field.
- Add the required attribute.
- Use meaningful button text.
- Use proper semantic HTML elements.
- Add placeholder text if necessary.
- Ensure keyboard accessibility.
- Validate user input.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

ANSWER:

1. Identify improper nesting and missing closing tags.

Problems:

- Missing <!DOCTYPE html>
- <h1> is not closed.
- <p> is not closed.
- Missing <meta charset="UTF-8">
- Missing viewport meta tag.
- No semantic HTML elements.

2. Analyze how different browsers may interpret this structure.

Different browsers may:

- Automatically insert missing closing tags.
- Render the page differently.
- Display inconsistent layouts.
- Produce validation errors.

3. Identify missing semantic elements.

Missing semantic elements include:

- <header>
- <main>
- <section>
- <footer>

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Company</title>
</head>
<body>
```

```
<header>
<h1>Welcome</h1>
</header>
<main>
<section>
<p>About us</p>
</section>
</main>
<footer>
<p>&copy; 2026 Company</p>
</footer>
</body>
</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

- Use valid HTML5 code.
- Validate HTML using the W3C Validator.
- Use semantic HTML elements.
- Test the website on multiple browsers.
- Use CSS reset or normalize stylesheets.
- Write standards-compliant CSS.
- Include the HTML5 DOCTYPE.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

ANSWER:

1. Identify issues in the form configuration.

Problems:

- Uses **GET** instead of **POST** for login credentials.
- Button type is **button** instead of **submit**.
- No labels for form fields.
- No input validation.
- Password is exposed in the URL when using GET.

2. Analyze why the HTTP request is not triggered.

The button is:

```
<button type="button">Login</button>
```

A button with type="button" **does not submit the form**.

Only

```
<button type="submit">
```

or JavaScript can trigger form submission.

3. Interpret the role of GET method in this scenario.

The **GET** method:

- Sends form data in the URL.
- Is suitable for searching or retrieving data.
- Is **not suitable for login forms** because usernames and passwords become visible in the browser URL.

Example:

```
/submit?username=amrita&password=12345
```

4. Propose a corrected implementation.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Login</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>User Login</h1>
```

```
</header>
<main>
<section>
<form action="/submit" method="POST">
<label for="username">Username:</label>
<input type="text" id="username" name="username" required>
<label for="password">Password:</label>
<input type="password" id="password" name="password" required>
<button type="submit">Login</button>
</form>
</section>
</main>
</body>
</html>
```

5. Suggest improvements for secure data transmission.

- Use the **POST** method instead of GET.
- Use **HTTPS** to encrypt communication.
- Validate input on both the client and server.
- Use strong password policies.
- Store passwords securely using hashing.
- Implement authentication and session management.
- Protect against SQL injection and Cross-Site Scripting (XSS).